STM32F103的流水灯点亮版本1(寄存器地址操作)

于 2026-09-22 19:08:50 发布

前言

这次作业是用 STM32F103C8T6 最小系统板，直接操作寄存器来控制 GPIO，实现一个三个 LED 的流水灯。之前用 STM32 基本都是调库函数，对寄存器这一层了解得很少，所以这次把自己从配时钟到烧录运行的整个过程记下来，也方便以后回看。

一、实验准备

用的东西就下面这些：

- STM32F103C8T6 最小系统板（Blue Pill）一块
- ST-Link 下载器一个
- 面包板一块
- 红、绿、蓝 LED 各一个
- 杜邦线若干
- Keil MDK 5

二、STM32F103C8T6 最小系统核心板

STM32F103C8T6 是一颗 Cortex-M3 内核的芯片，主频最高 72MHz，64KB Flash、20KB RAM，LQFP48 封装。它的 GPIO 有 PA、PB、PC 几组，其中 C 口在 48 脚封装下只有 PC13、PC14、PC15 三个脚。

三、硬件接线

三个外接 LED 分别接在 PA0、PB0、PC14 上，接法是「低电平点亮」：LED 正极接 3.3V，负极到 GPIO 引脚。引脚输出低电平的时候，3.3V 到引脚之间有电流流过，LED 才亮；引脚拉高以后 LED 两端没有电压差，就灭了。
PA0 | 红色 LED | 正极 3.3V，负极经电阻到 PA0
PB0 | 绿色 LED | 正极 3.3V，负极经电阻到 PB0
PC14 | 蓝色 LED | 正极 3.3V，负极经电阻到 PC14

ST-Link 和板子之间接四根线：SWDIO、SWCLK、GND、3.3V。

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/166369229
作者主页：https://blog.csdn.net/2503_90536812

四、GPIO 和 RCC 寄存器介绍

这次用到的寄存器，都是我从 STM32 中文参考手册里翻出来的。下面记一下每个寄存器我是怎么选的、设成了什么。

4.1 开时钟：RCC_APB2ENR

GPIOA、GPIOB、GPIOC 三个端口都挂在 APB2 总线上，用之前得先给它们开时钟，不然引脚不会工作。我在手册里找到时钟使能寄存器 RCC_APB2ENR，它的 bit2、bit3、bit4 分别对应 A、B、C 三个端口的时钟开关，于是设成：

```
RCC_APB2ENR |= (1u << 2) | (1u << 3) | (1u << 4);  // 同时打开 A、B、C 三个端口的时钟
```

0x4002 3000 - 0x4002 33FF	CRC		参见3.4.4节
0x4002 2000 - 0x4002 23FF	闪存存储器接口		
0x4002 1400 - 0x4002 1FFF	保留		
0x4002 1000 - 0x4002 13FF	复位和时钟控制(RCC)	AHB	参见6.3.11节

4.2 配引脚：CRL / CRH

接下来要把 PA0、PB0、PC14 三个脚配成输出。每个脚在配置寄存器里占 4 位，0~7 号脚在 CRL，8~15 号脚在 CRH。

必须以字(32位)的方式操作这些外设寄存器。

8.2.1 端口配置低寄存器(GPIOx_CRL) (x=A..E)

偏移地址：0x00

复位值：0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:30 27:26 23:22 19:18 15:14 11:10 7:6 3:2	**CNFy[1:0]**：端口x配置位(y = 0…7) (Port x configuration bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 在输入模式(MODE[1:0]=00)： 00：模拟输入模式 01：浮空输入模式(复位后的状态) 10：上拉/下拉输入模式 11：保留 在输出模式(MODE[1:0]>00)： 00：通用推挽输出模式 01：通用开漏输出模式 10：复用功能推挽输出模式 11：复用功能开漏输出模式
位29:28 25:24 21:20 17:16 13:12 9:8, 5:4 1:0	**MODEy[1:0]**：端口x的模式位(y = 0…7) (Port x mode bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 00：输入模式(复位后的状态) 01：输出模式，最大速度10MHz 10：输出模式，最大速度2MHz 11：输出模式，最大速度50MHz

8.2.2 端口配置高寄存器(GPIOx_CRH) (x=A..E)

偏移地址：0x04

复位值：0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:30 27:26 23:22 19:18 15:14 11:10 7:6 3:2	**CNFy[1:0]**：端口x配置位(y = 8…15) (Port x configuration bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 在输入模式(MODE[1:0]=00)： 00：模拟输入模式 01：浮空输入模式(复位后的状态) 10：上拉/下拉输入模式 11：保留 在输出模式(MODE[1:0]>00)： 00：通用推挽输出模式 01：通用开漏输出模式 10：复用功能推挽输出模式 11：复用功能开漏输出模式
位9:28 25:24 21:20 17:16 13:12 9:8, 5:4 1:0	**MODEy[1:0]**：端口x的模式位(y = 8…15) (Port x mode bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 00：输入模式(复位后的状态) 01：输出模式，最大速度10MHz 10：输出模式，最大速度2MHz 11：输出模式，最大速度50MHz

作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/166369229
作者主页：https://blog.csdn.net/2503_90536812

0x4001 1000 - 0x4001 13FF	GPIO端口C
0X4001 0C00 - 0x4001 0FFF	GPIO端口B
0x4001 0800 - 0x4001 0BFF	GPIO端口A

```
GPIOA_CRL &= ~(0xFu << 0);
GPIOA_CRL |=  (0x3u << 0);

GPIOB_CRL &= ~(0xFu << 0);
GPIOB_CRL |=  (0x3u << 0);

GPIOC_CRH &= ~(0xFu << 24);
GPIOC_CRH |=  (0x2u << 24);
```

4.3 控制亮灭：BSRR / BRR

引脚配成输出后，用 BSRR 和 BRR 决定它出高电平还是低电平：BSRR 的低 16 位写 1 是拉高，BRR 写 1 是拉低。

8.2.5 端口位设置/清除寄存器(GPIOx_BSRR) (x=A..E)

地址偏移：0x10

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位31:16	**BRy**: 清除端口x的位y (y = 0…15) (Port x Reset bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：清除对应的ODRy位为0 注：如果同时设置了BSy和BRy的对应位，BSy位起作用。
位15:0	**BSy**: 设置端口x的位y (y = 0…15) (Port x Set bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：设置对应的ODRy位为1

8.2.6 端口位清除寄存器(GPIOx_BRR) (x=A..E)

地址偏移：0x14

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

参照2009年12月 RM0008 Reference Manual 英文第10版

本译文仅供参考，如有翻译错误，请以英文原稿为准。请读者随时注意在ST网站下载更新版本

位31:16	保留。
位15:0	**BRy**: 清除端口x的位y (y = 0…15) (Port x Reset bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：清除对应的ODRy位为0

https://blog.csdn.net/2503_90536812/article/details/166369229

我的 LED 是「正极接 3.3V、负极接引脚」的低电平点亮，所以点亮用 BRR 拉低，熄灭用 BSRR 拉高。

4.4 延时：SysTick

每个灯要亮 1 秒，需要一个延时。我选了芯片内核自带的 SysTick 定时器，没用 for 循环——for 循环延时的时间和编译器优化有关，不精确。SysTick 在 8MHz 时钟下 1ms 正好数 8000 个，所以 delay_ms(1000) 就是准确的 1 秒。

五、三个 LED 流水灯

5.1 程序设计思路

流水灯就是三个灯轮流亮：红灯亮 1 秒 → 绿灯亮 1 秒 → 蓝灯亮 1 秒，一直循环。用状态来表示就是：
状态一：1 0 0（红亮）
状态二：0 1 0（绿亮）
状态三：0 0 1（蓝亮）

5.2 寄存器配置

一共三步：

1. 打开时钟：`RCC_APB2ENR` 把 bit2、bit3、bit4 置 1。
2. 配引脚：PA0、PB0 在 CRL 里，配 50MHz 推挽输出（0x3）；PC14 在 CRH 里，配 2MHz 推挽输出（0x2）。
3. 主循环里用 BRR/BSRR 控制亮灭

5.3 完整代码

一、寄存器地址定义

```
/* RCC 时钟控制寄存器 */
#define RCC_BASE        0x40021000U
#define RCC_APB2ENR     (*(volatile unsigned int *)(RCC_BASE + 0x18))  /* APB2 外设时钟使能 */

/* GPIO 端口基地址 */
#define GPIOA_BASE      0x40010800U
#define GPIOB_BASE      0x40010C00U
#define GPIOC_BASE      0x40011000U
```

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/166369229
作者主页：https://blog.csdn.net/2503_90536812

```
/* GPIOA 的寄存器 */
#define GPIOA_CRL   (*(volatile unsigned int *)(GPIOA_BASE + 0x00))  /* 低配置寄存器(引脚0~7) */
#define GPIOA_CRH   (*(volatile unsigned int *)(GPIOA_BASE + 0x04))  /* 高配置寄存器(引脚8~15) */
#define GPIOA_BSRR  (*(volatile unsigned int *)(GPIOA_BASE + 0x10))  /* 置位/复位寄存器 */
#define GPIOA_BRR   (*(volatile unsigned int *)(GPIOA_BASE + 0x14))  /* 复位寄存器(写1拉低) */

/* GPIOB 的寄存器 */
#define GPI
#define GPIOB_BSRR  (*(volatile unsigned int *)(GPIOB_BASE + 0x10))
#define GPIOB_BRR   (*(volatile unsigned int *)(GPIOB_BASE + 0x14))

/* GPIOC 的寄存器 */
#define GPIOC_CRL   (*(volatile unsigned int *)(GPIOC_BASE + 0x00))
#define GPIOC_CRH   (*(volatile unsigned int *)(GPIOC_BASE + 0x04))
#define GPIOC_BSRR  (*(volatile unsigned int *)(GPIOC_BASE + 0x10))
#define GPIOC_BRR   (*(volatile unsigned int *)(GPIOC_BASE + 0x14))

/* SysTick 寄存器（内核固定地址） */
#define SYSTICK_CTRL  (*(volatile unsigned int *)0xE000E010)
#define SYSTICK_LOAD  (*(volatile unsigned int *)0xE000E014)
#define SYSTICK_VAL   (*(volatile unsigned int *)0xE000E018)
```

二、延时函数

```
/* HSI 8MHz 时钟，SysTick 时钟 = 8MHz，1ms 数 8000 个 */
void delay_ms(unsigned int ms)
{
    SYSTICK_CTRL = 0x5u;                    /* bit0使能, bit2用HCLK时钟 */
    while ((SYSTICK_CTRL & 0x10000u) == 0u);  /* 等 bit16 COUNTFLAG=1 */
    SYSTICK_CTRL = 0u;                      /* 关闭 SysTick */
}
```

三、SystemInit 函数

```
/* 启动文件进入 main 前会调用，本实验用默认 HSI 时钟，留空即可 */
void SystemInit(void)
{
    /* 不需要配置时钟 */
}
```

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/166369229
作者主页：https://blog.csdn.net/2503_90536812

四、主函数

```
int main(vo
    GPIOB_CRL |=  (0x3u << 0);   /* PB0：50MHz 推挽输出 */

    GPIOC_CRH &= ~(0xFu << 24);
    GPIOC_CRH |=  (0x2u << 24);  /* PC14：2MHz 推挽输出 */

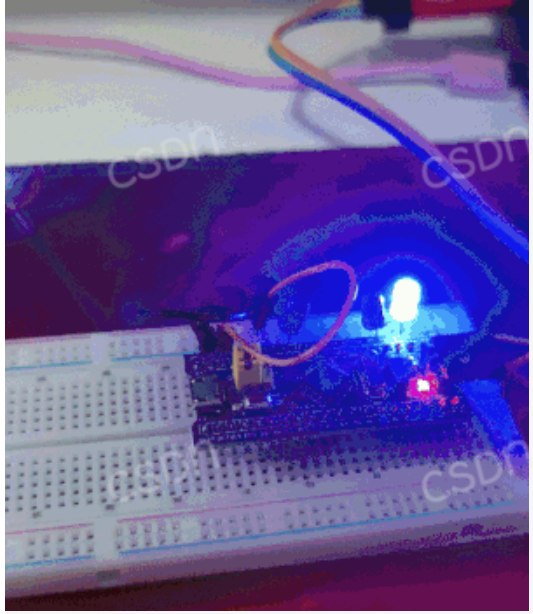
    /* 第3步：主循环，红 → 绿 → 蓝 轮流点亮（低电平点亮） */
    while (1)
    {
        /* 红灯亮 1 秒，绿、蓝灭 */
        GPIOA_BRR  = (1u << 0);         /* PA0 置低 → 红亮 */
        GPIOB_BSRR = (1u << 0);         /* PB0 置高 → 绿灭 */
        GPIOC_BSRR = (1u << 14);        /* PC14 置高 → 蓝灭 */
        delay_ms(1000);

        GPIOC_BRR  = (1u << 14);        /* 蓝亮 */
        delay_ms(1000);
    }
}
```

5.4 运行结果

代码在 Keil 里编译，0 Error、0 Warning。用 ST-Link 下载 到板子后，三个 LED 依次循环点亮，每个亮 1 秒。

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/166369229
作者主页：https://blog.csdn.net/2503_90536812



六、遇到的问题

1. 中文路径导致编译报错。Windows 用户名是中文，工程最初放在中文路径下，Keil AC6 编译启动文件时报 Unable to convert command-line to UTF-16。查了才知道是汇编器 spawn 子进程时命令行里带了中文路径、转 UTF-16 失败，而.c文件不受影响。最后把启动文件改成 GNU 汇编语法、工程移到纯英文路径解决。

2. 高电平和低电平点亮搞反。一开始代码按「引脚拉高 LED 亮」写，结果板子上一堆灯乱亮——一直有两个灯同时亮、每个灯亮两秒。后来才发现我的 LED 是「正极接 3.3V、负极接引脚」的低电平点亮，引脚拉低才亮。把「点亮」改成 BRR 拉低、「熄灭」改成 BSRR 拉高就好了。

七、总结

这次从最底层的寄存器入手把流水灯跑通了，对「用外设之前先开时钟」「一个引脚 4 位配置」「BSRR 置高、BRR拉低」这些概念有了实感，中间踩的中文路径、高低电平反了这几个坑，都是自己一点一点排出来的，记下来以后不会再犯。

标签
#github #keil5 #stm32 #c8t6

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/166369229
作者主页：https://blog.csdn.net/2503_90536812